

Sprawozdanie Mateusz Hyps – lab 6

1. Algorytmy ByteRun i RLE czytają ciągi symboli jeden po drugim, dlatego wywłaszczamy dane za pomocą funkcji `flatten()`. Przed kompresją zapisujemy jednak ich kształt (`shape`), np. przy użyciu `np.concatenate((shape_info, compressed_data))`. Dzięki temu w procesie dekompresji możemy odtworzyć dokładne wymiary danych. Jeśli chodzi o proces kompresji, tworzymy bufor wynikowy, np. za pomocą `np.zeros()`, który następnie nadpisujemy i na końcu ucinamy niewykorzystane pozostałości.
2. Oba algorytmy działają poprawnie — dane przed kompresją i po dekompresji nie ulegają zmianie. Zostało to sprawdzone w kodzie za pomocą funkcji `np.array_equal(img_int, decoded_rle)`, która porównuje obraz wejściowy z obrazem po dekompresji. Dodatkowo poprawność działania została zweryfikowana na zestawie danych testowych:

```
“test_cases = [  
    np.array([1,1,1,1,2,1,1,1,1,2,1,1,1]),  
    np.array([1,2,3,1,2,3,1,2,3]),  
    np.array([5,1,5,1,5,5,1,1,5,5,1,1,5]),  
    np.array([-1,-1,-1,-5,-5,-3,-4,-2,1,2,2,1]),  
    np.zeros((1,520)),  
    np.arange(0,521,1),  
    np.eye(7),  
    np.dstack([np.eye(7), np.eye(7), np.eye(7)]),  
    np.ones((1,1,1,1,1,1,1,10))  
]”
```

Testując algorytmy na powyższych przypadkach, również otrzymujemy identyczne dane przed kompresją i po dekompresji.

3. Obrazy techniczne i dokumenty (`rysunek-tech.png`, `cv.png`): Oba algorytmy poradziły sobie świetnie. Rysunki i formularze tekstowe posiadają duże, jednolite obszary (np. dużo białego tła), więc znaki mocno się powtarzają. Dla rysunku technicznego skuteczność kompresji (PR) to ok. 37% dla RLE i niecałe 32% dla ByteRun. Dla CV wyniki są podobne (ByteRun wypadł nieco lepiej, osiągając ok. 33%).

Zdjęcie kolorowe (color.png): W tym przypadku występuje skrajnie niesprzyjająca sytuacja dla obu algorytmów, w której dane po kompresji okazują się większe niż dane wejściowe. Wynika to z charakterystyki obrazów kolorowych — rzadko pojawiają się w nich długie ciągi identycznych pikseli, a dominują detale, szумы oraz płynne przejścia tonalne. Algorytm RLE wypada w tym scenariuszu bardzo niekorzystnie, zwiększając rozmiar danych niemal dwukrotnie (około 199% oryginalnego rozmiaru). Dzieje się tak dlatego, że każdy pojedynczy, niepowtarzający się symbol zapisywany jest jako para: licznik (równy 1) oraz wartość symbolu, co znacząco zwiększa ilość danych. Algorytm ByteRun radzi sobie wyraźnie lepiej, osiągając rozmiar zbliżony do oryginału (około 101%). Jest to możliwe dzięki zastosowaniu mechanizmu pozwalającego efektywnie kodować sekwencje niepowtarzających się symboli bez konieczności dodawania nadmiarowego licznika dla każdego z nich.

Output z konsoli pokazujący wyniki:

```
(venv) mathyp@Mateusz:~/sm_2026/lab_6$ python SM_Lab_06.py
```

```
----- Test zdjęcia rysunek-tech.png -----
```

```
Identyczność z oryginałem (RLE): True
```

```
Identyczność z oryginałem (ByteRun): True
```

```
RLE: stopień kompresji: 2.6995, czyli 37.04 %
```

```
ByteRun: stopień kompresji: 3.1443, czyli 31.80 %
```

```
----- Test zdjęcia cv.png -----
```

```
Identyczność z oryginałem (RLE): True
```

```
Identyczność z oryginałem (ByteRun): True
```

```
RLE: stopień kompresji: 1.8728, czyli 53.39 %
```

```
ByteRun: stopień kompresji: 2.9914, czyli 33.43 %
```

```
----- Test zdjęcia color.png -----
```

```
Identyczność z oryginałem (RLE): True
```

```
Identyczność z oryginałem (ByteRun): True
```

```
RLE: stopień kompresji: 0.5019, czyli 199.25 %
```

```
ByteRun: stopień kompresji: 0.9896, czyli 101.06 %
```